

Relaciones en Sequelize

1. Introducción

En el desarrollo de aplicaciones con Sequelize, las asociaciones permiten modelar las relaciones entre entidades de una base de datos relacional. Sequelize soporta los tres tipos estándar de relaciones:

- Uno a Uno (1:1)
- Uno a Muchos (1:N)
- Muchos a Muchos (N:M)

Para implementarlas, dispone de cuatro métodos principales:

- `hasOne`
- `belongsTo`
- `hasMany`
- `belongsToMany`

Cada método indica dónde se ubica la clave foránea (FK) y cómo se generan las funciones auxiliares para interactuar con los datos relacionados.

2. Tipos de relaciones y características

2.1. `hasOne` (tiene uno)

Qué significa: "Este modelo TIENE UNO de otro modelo"

Ejemplo: User tiene un Profile

```
UserModel.hasOne(ProfileModel, { foreignKey: "user_id", as: "profile" })
```

Dónde va la clave foránea: En la tabla Profiles. Se crea la columna **user_id** en la tabla Profiles, que apunta al id de la tabla Users.

2.2. `belongsTo` (pertenece a)

Qué significa: "Este modelo PERTENECE A otro modelo"

Ejemplo: Profile pertenece a un User

```
ProfileModel.belongsTo(UserModel, { foreignKey: "user_id", as: "user" })
```

Dónde va la clave foránea: En la tabla Profiles. Se crea la columna **user_id** en la tabla Profiles, que apunta al **id** de la tabla Users.

2.3. hasMany (tiene muchos)

Qué significa: "Este modelo TIENE MUCHOS de otro modelo"

Ejemplo: User tiene muchas Tasks

```
UserModel.hasMany(TaskModel, { foreignKey: "user_id", as: "tasks" })
```

Dónde va la clave foránea: En la tabla Tasks. Se crea la columna **user_id** en la tabla Tasks, que apunta al **id** de la tabla Users.

2.4. belongsToMany (muchos a muchos)

Qué significa: "Muchos de este modelo se relacionan con muchos de otro modelo"

Ejemplo: User puede tener muchos Roles, y Role puede pertenecer a muchos Users

```
UserModel.belongsToMany(RoleModel, { through: UserRoleModel,
foreignKey: "user_id", as: "roles" })
RoleModel.belongsToMany(UserModel, { through: UserRoleModel,
foreignKey: "role_id", as: "users" })
```

Dónde van las claves foráneas: En una tabla intermedia **UserRole** con dos columnas: **user_id** (apunta al **id** de Users) y **role_id** (apunta al **id** de Roles).

Reglas simples para recordar:

- hasOne + belongsTo = Relación 1:1 (la FK va donde dice belongsTo)
- hasMany + belongsTo = Relación 1:N (la FK va donde dice belongsTo)
- belongsToMany + belongsToMany = Relación N:M (necesita tabla intermedia)

3. Consideraciones técnicas en Sequelize

3.1. Claves foráneas y alias

- El parámetro **foreignKey** permite definir el nombre exacto de la clave foránea.
- El parámetro **as** define un alias para la relación, que debe usarse tanto en consultas como en las funciones generadas por Sequelize.
- Todas las claves foráneas que contengan dos o más palabras deben escribirse en minúsculas y en formato **snake_case** (por ejemplo, `user_id`, `role_id`). Esta convención garantiza uniformidad en el proyecto y evita inconsistencias entre el modelo y la base de datos.

3.2. Direccionalidad de las relaciones

- Sequelize solo reconoce la relación desde el modelo donde se define.
- Para permitir consultas en ambos sentidos, se recomienda declarar las relaciones en pares (hasOne con belongsTo, hasMany con belongsTo, dos belongsToMany).

3.3. Carga de datos asociados

Eager Loading: obtiene los datos relacionados en la misma consulta mediante la opción include.

```
const users = await UserModel.findAll({
  include: [{ model: TaskModel, as: "tasks" }]
});
console.log(users[0].tasks); // Las tareas del usuario ya están disponibles
```

3.4. Restricciones y borrado en cascada

- Por defecto, en relaciones 1:1 y 1:N, ON DELETE es SET NULL y ON UPDATE es CASCADE.
- En relaciones N:M, por defecto, ON DELETE y ON UPDATE son CASCADE.
- Estas opciones pueden modificarse según las reglas de integridad necesarias.

4. Ubicación de las relaciones y prevención de dependencias circulares

En proyectos con Sequelize, las relaciones deben definirse en un punto central y después de que todos los modelos hayan sido creados. Esto evita que un modelo intente importar directamente a otro para definir su asociación, lo que generaría una dependencia circular.

El orden de trabajo recomendado es:

1. Definir todos los modelos con sus atributos y opciones, sin relaciones.
2. En un único módulo o sección, establecer todas las asociaciones entre modelos.
3. Asegurar que las claves foráneas utilicen la convención **snake_case** en minúsculas cuando el nombre contenga dos palabras, manteniendo uniformidad en todo el proyecto.

Este enfoque garantiza que cada modelo pueda acceder a sus relaciones sin producir errores de carga y que las consultas con **include** funcionen correctamente en ambos sentidos.

5. Buenas prácticas

- Definir siempre alias claros y consistentes para evitar ambigüedades.
- Utilizar **through: { attributes: [] }** en relaciones N:M para omitir columnas de la tabla intermedia en los resultados.
- Configurar **allowNull** y **onDelete** según las necesidades de integridad referencial.
- Agregar índices únicos en las tablas de unión para evitar duplicados en las relaciones

N:M.

- Declarar las relaciones en pares para habilitar la navegación en ambos sentidos.